



ShopVibe – Tài Liệu Đặc Tả Frontend

Stack: React 18 + TypeScript + Vite + React Query + Redux Toolkit + Axios + Zod + React Hook Form + TailwindCSS

Base URL: http://localhost:8080/api/v1

Auth: JWT Bearer Token (HS512) + Refresh Token (UUID)

Roles: ROLE_USER, ROLE_ADMIN

1. TỔNG QUAN HỆ THỐNG

ShopVibe là nền tảng thương mại điện tử gồm hai phần:

- **Customer App** – Khách hàng duyệt sản phẩm, giỏ hàng, đặt hàng, đánh giá.
- **Admin Dashboard** – Quản trị sản phẩm, danh mục, đơn hàng, người dùng, thống kê doanh thu.

Modules chính: Auth · Categories · Products · Cart · Orders · Reviews · Users · Statistics

2. PHÂN TÍCH API

2.1 Danh sách toàn bộ Endpoint

#	Method	Endpoint	Auth	Role	Mô tả
1	POST	/auth/register		-	Đăng ký tài khoản
2	POST	/auth/login		-	Đăng nhập
3	POST	/auth/logout		USER/ADMIN	Đăng xuất
4	POST	/auth/refresh-token		-	Làm mới access token
5	POST	/auth/introspect		-	Kiểm tra token hợp lệ
6	PUT	/auth/change-password		USER/ADMIN	Đổi mật khẩu
7	GET	/categories		-	Lấy danh sách danh mục
8	GET	/categories/:id		-	Lấy danh mục theo ID
9	POST	/		ADMIN	Tạo danh mục

#	Method	Endpoint	Auth	Role	Mô tả
		categories			
10	PUT	/categories/:id		ADMIN	Cập nhật danh mục
11	DELETE	/categories/:id		ADMIN	Xóa danh mục
12	GET	/products?page&size&sort		-	Lấy danh sách sản phẩm (phân trang)
13	GET	/products/:id		-	Lấy sản phẩm theo ID
14	GET	/products/search?keyword&page&size&sort		-	Tìm kiếm sản phẩm
15	POST	/products		ADMIN	Tạo sản phẩm
16	PUT	/products/:id		ADMIN	Cập nhật sản phẩm
17	DELETE	/products/:id		ADMIN	Xóa sản phẩm
18	POST	/products/:id/images		ADMIN	Upload ảnh sản phẩm (multipart)
19	DELETE	/products/:id/images/:imageId		ADMIN	Xóa ảnh sản phẩm
20	GET	/cart		USER	Xem giỏ hàng
21	POST	/cart/items		USER	Thêm vào giỏ hàng
22	PUT	/cart/items/:itemId		USER	Cập nhật số lượng
23	DELETE	/cart/items/:itemId		USER	Xóa sản phẩm khỏi giỏ
24	DELETE	/cart/		USER	Xóa toàn bộ

#	Method	Endpoint	Auth	Role	Mô tả
		clear			giỏ hàng
25	POST	/orders		USER	Tạo đơn hàng từ giỏ
26	GET	/orders?page&size&status&from&to		USER	Lấy đơn hàng của tôi
27	GET	/orders/:id		USER	Chi tiết đơn hàng
28	PUT	/orders/:id/cancel		USER	Hủy đơn hàng
29	GET	/orders/admin/all?page&size&status&from&to		ADMIN	Tất cả đơn hàng (admin)
30	PUT	/orders/admin/:id/status		ADMIN	Cập nhật trạng thái đơn
31	GET	/reviews/products/:productId		-	Lấy đánh giá theo sản phẩm
32	POST	/reviews/products/:productId		USER	Tạo đánh giá
33	PUT	/reviews/:reviewId		USER	Cập nhật đánh giá
34	DELETE	/reviews/:reviewId		USER	Xóa đánh giá
35	GET	/users/me		USER/ADMIN	Thông tin bản thân
36	PUT	/users/me		USER/ADMIN	Cập nhật thông tin
37	GET	/users/admin/all		ADMIN	Tất cả người dùng
38	PUT	/users/admin/:id/lock		ADMIN	Khóa tài khoản
39	PUT	/users/admin/:id/unlock		ADMIN	Mở khóa tài khoản

#	Method	Endpoint	Auth	Role	Mô tả
40	GET	/admin/statistics/revenue?from&to		ADMIN	Doanh thu theo khoảng thời gian
41	GET	/admin/statistics/top-products?limit		ADMIN	Top sản phẩm bán chạy
42	GET	/admin/statistics/top-customers?limit		ADMIN	Top khách hàng
43	GET	/admin/statistics/overview		ADMIN	Tổng quan dashboard

2.2 Nhóm theo Module

Module	Endpoints
auth	1-6
categories	7-11
products	12-19
cart	20-24
orders	25-30
reviews	31-34
users	35-39
statistics	40-43

2.3 Luồng nghiệp vụ chính

[Khách hàng]

Register/Login → Browse Products → Search/Filter → View Detail
 → Add to Cart → Update Quantity → Checkout (Create Order)
 → View Orders → Cancel Order → Leave Review

[Admin]

Login → Dashboard Overview → Manage Products (CRUD + Images)
 → Manage Categories (CRUD) → Manage Orders (Filter + Update Status)
 → Manage Users (Lock/Unlock) → View Statistics (Revenue, Top Products, Top Customers)

2.4 Order Status Flow

PENDING → CONFIRMED → SHIPPING → DELIVERED

↓
CANCELLED (chỉ từ PENDING)

3. CẤU TRÚC THƯ MỤC DỰ ÁN

```
src/
├── app/
│   ├── store.ts           # Redux store
│   └── hooks.ts          # useAppDispatch, useAppSelector
├── assets/               # Images, fonts, icons
├── components/          # Shared UI Components
│   ├── ui/
│   │   ├── Button.tsx
│   │   ├── Input.tsx
│   │   ├── Modal.tsx
│   │   ├── Table.tsx
│   │   ├── Pagination.tsx
│   │   ├── Badge.tsx
│   │   ├── Spinner.tsx
│   │   ├── StarRating.tsx
│   │   ├── ImageUpload.tsx
│   │   └── DateRangePicker.tsx
│   └── common/
│       ├── ErrorBoundary.tsx
│       ├── ProtectedRoute.tsx
│       ├── AdminRoute.tsx
│       └── PageTitle.tsx
├── constants/
│   ├── api.ts            # API_BASE_URL, endpoints
│   ├── queryKeys.ts      # React Query keys
│   └── app.ts            # ORDER_STATUS, SORT_OPTIONS, ROLES
├── features/
│   ├── auth/
│   │   ├── components/
│   │   │   ├── LoginForm.tsx
│   │   │   └── RegisterForm.tsx
│   │   ├── hooks/
│   │   │   └── useAuth.ts
│   │   ├── store/
│   │   │   └── authSlice.ts
│   │   ├── services/
│   │   │   └── authService.ts
│   │   └── types/
│   │       └── auth.types.ts
│   └── products/
│       ├── components/
│       │   ├── ProductCard.tsx
│       │   ├── ProductGrid.tsx
│       │   ├── ProductDetail.tsx
│       │   ├── ProductForm.tsx
│       │   ├── ProductImageGallery.tsx
│       │   └── ProductSearch.tsx
│       ├── hooks/
│       │   ├── useProducts.ts
│       │   └── useProductDetail.ts
│       └── services/
```

```
├── productService.ts
├── types/
│   └── product.types.ts
├── categories/
│   ├── components/
│   │   ├── CategoryTree.tsx
│   │   └── CategoryForm.tsx
│   ├── hooks/
│   │   └── useCategories.ts
│   ├── services/
│   │   └── categoryService.ts
│   └── types/
│       └── category.types.ts
├── cart/
│   ├── components/
│   │   ├── CartDrawer.tsx
│   │   ├── CartItem.tsx
│   │   └── CartSummary.tsx
│   ├── hooks/
│   │   └── useCart.ts
│   ├── store/
│   │   └── cartSlice.ts
│   ├── services/
│   │   └── cartService.ts
│   └── types/
│       └── cart.types.ts
├── orders/
│   ├── components/
│   │   ├── OrderList.tsx
│   │   ├── OrderCard.tsx
│   │   ├── OrderDetail.tsx
│   │   ├── OrderStatusBadge.tsx
│   │   └── CheckoutForm.tsx
│   ├── hooks/
│   │   └── useOrders.ts
│   ├── services/
│   │   └── orderService.ts
│   └── types/
│       └── order.types.ts
├── reviews/
│   ├── components/
│   │   ├── ReviewList.tsx
│   │   ├── ReviewForm.tsx
│   │   └── ReviewItem.tsx
│   ├── hooks/
│   │   └── useReviews.ts
│   ├── services/
│   │   └── reviewService.ts
│   └── types/
│       └── review.types.ts
├── users/
│   ├── components/
│   │   ├── ProfileForm.tsx
│   │   └── ChangePasswordForm.tsx
│   ├── hooks/
│   │   └── useUser.ts
│   ├── services/
│   │   └── userService.ts
│   └── types/
```

```

├── user.types.ts
├── statistics/
│   ├── components/
│   │   ├── RevenueChart.tsx
│   │   ├── TopProductsTable.tsx
│   │   ├── TopCustomersTable.tsx
│   │   └── OverviewCards.tsx
│   ├── hooks/
│   │   └── useStatistics.ts
│   ├── services/
│   │   └── statisticsService.ts
│   └── types/
│       └── statistics.types.ts
├── hooks/                                # Global shared hooks
│   ├── useDebounce.ts
│   └── useLocalStorage.ts
├── layouts/
│   ├── MainLayout.tsx                  # Header + Footer (customer)
│   ├── AdminLayout.tsx                 # Sidebar + Topbar (admin)
│   └── AuthLayout.tsx                  # Centered card (login/register)
├── pages/
│   ├── public/
│   │   ├── HomePage.tsx
│   │   ├── ProductListPage.tsx
│   │   ├── ProductDetailPage.tsx
│   │   └── SearchResultPage.tsx
│   ├── auth/
│   │   ├── LoginPage.tsx
│   │   └── RegisterPage.tsx
│   ├── user/
│   │   ├── CartPage.tsx
│   │   ├── CheckoutPage.tsx
│   │   ├── OrderListPage.tsx
│   │   ├── OrderDetailPage.tsx
│   │   └── ProfilePage.tsx
│   └── admin/
│       ├── DashboardPage.tsx
│       ├── ProductManagePage.tsx
│       ├── ProductFormPage.tsx
│       ├── CategoryManagePage.tsx
│       ├── OrderManagePage.tsx
│       └── UserManagePage.tsx
├── routes/
│   ├── index.tsx
│   ├── publicRoutes.tsx
│   ├── userRoutes.tsx
│   └── adminRoutes.tsx
├── services/
│   └── api/
│       ├── axiosClient.ts              # Axios instance + interceptors
│       └── endpoints.ts
├── types/
│   ├── common.types.ts                # PaginationResponse, ApiResponse,
│   └── index.ts                        ErrorResponse
├── Error
├── ErrorResponse
└── utils/

```

- formatCurrency.ts
- formatDate.ts
- tokenUtils.ts

4. TYPESCRIPT TYPES

```
// — types/common.types.ts —————  
  
export interface ApiResponse<T> {  
  code: number;  
  message: string;  
  result: T;  
}  
  
export interface PaginationResponse<T> {  
  content: T[];  
  totalElements: number;  
  totalPages: number;  
  size: number;  
  number: number;           // current page (0-indexed)  
  first: boolean;  
  last: boolean;  
  empty: boolean;  
}  
  
export interface ErrorResponse {  
  code: number;  
  message: string;  
}  
  
// — features/auth/types/auth.types.ts —————  
  
export interface LoginRequest {  
  email: string;  
  password: string;  
}  
  
export interface RegisterRequest {  
  fullName: string;  
  email: string;  
  password: string;  
  phone: string;  
}  
  
export interface AuthResponse {  
  accessToken: string;  
  refreshToken: string;  
}  
  
export interface RefreshTokenRequest {  
  refreshToken: string;  
}  
  
export interface LogoutRequest {  
  accessToken: string;  
  refreshToken: string;  
}  
  
export interface IntrospectRequest {  
  accessToken: string;  
}
```

```
export interface IntrospectResponse {
  valid: boolean;
}

export interface ChangePasswordRequest {
  email: string;
  oldPassword: string;
  newPassword: string;
}

// — features/categories/types/category.types.ts —————

export interface Category {
  id: number;
  name: string;
  parentCategoryId: number | null;
  children?: Category[];
}

export interface CreateCategoryRequest {
  name: string;
  parentCategoryId?: number | null;
}

export type UpdateCategoryRequest = CreateCategoryRequest;

// — features/products/types/product.types.ts —————

export interface ProductImage {
  id: number;
  imageUrl: string;
}

export interface Product {
  id: number;
  name: string;
  description: string;
  price: number;
  stockQuantity: number;
  categoryId: number;
  categoryName?: string;
  images: ProductImage[];
  createdAt: string;
  updatedAt: string;
}

export interface CreateProductRequest {
  name: string;
  description: string;
  price: number;
  stockQuantity: number;
  categoryId: number;
}

export type UpdateProductRequest = CreateProductRequest;

export interface ProductQueryParams {
  page?: number;
  size?: number;
  sort?: 'newest' | 'oldest' | 'price_asc' | 'price_desc';
  keyword?: string;
}
```

```
// — features/cart/types/cart.types.ts —————
```

```
export interface CartItem {  
  id: number;  
  productId: number;  
  productName: string;  
  productImage?: string;  
  price: number;  
  quantity: number;  
  subtotal: number;  
}
```

```
export interface Cart {  
  items: CartItem[];  
  totalItems: number;  
  totalPrice: number;  
}
```

```
export interface AddToCartRequest {  
  productId: number;  
  quantity: number;  
}
```

```
export interface UpdateCartItemRequest {  
  quantity: number;  
}
```

```
// — features/orders/types/order.types.ts —————
```

```
export type OrderStatus =  
  | 'PENDING'  
  | 'CONFIRMED'  
  | 'SHIPPING'  
  | 'DELIVERED'  
  | 'CANCELLED';
```

```
export type PaymentMethod = 'COD' | 'BANK_TRANSFER' | 'MOMO';
```

```
export interface OrderItem {  
  id: number;  
  productId: number;  
  productName: string;  
  productImage?: string;  
  price: number;  
  quantity: number;  
  subtotal: number;  
}
```

```
export interface Order {  
  id: number;  
  userId: number;  
  shippingAddress: string;  
  note?: string;  
  paymentMethod: PaymentMethod;  
  status: OrderStatus;  
  items: OrderItem[];  
  totalPrice: number;  
  createdAt: string;  
  updatedAt: string;  
}
```

```
export interface CreateOrderRequest {  
  shippingAddress: string;  
  note?: string;  
}
```

```

    paymentMethod: PaymentMethod;
}

export interface CancelOrderRequest {
  reason: string;
}

export interface UpdateOrderStatusRequest {
  newStatus: OrderStatus;
}

export interface OrderQueryParams {
  page?: number;
  size?: number;
  status?: OrderStatus;
  from?: string;    // format: "dd-MM-yyyy"
  to?: string;      // format: "dd-MM-yyyy"
}

// — features/reviews/types/review.types.ts —————

export interface Review {
  id: number;
  userId: number;
  userFullName: string;
  productId: number;
  rating: number;    // 1-5
  comment: string;
  createdAt: string;
  updatedAt: string;
}

export interface CreateReviewRequest {
  rating: number;
  comment: string;
}

export type UpdateReviewRequest = CreateReviewRequest;

// — features/users/types/user.types.ts —————

export type UserRole = 'ROLE_USER' | 'ROLE_ADMIN';

export interface User {
  id: number;
  fullName: string;
  email: string;
  phone: string;
  address?: string;
  role: UserRole;
  locked: boolean;
  createdAt: string;
}

export interface UpdateUserRequest {
  fullName: string;
  phone: string;
  address?: string;
}

// — features/statistics/types/statistics.types.ts —————

export interface RevenueDataPoint {
  date: string;

```

```

    revenue: number;
    orderCount: number;
}

export interface RevenueResponse {
  data: RevenueDataPoint[];
  totalRevenue: number;
  totalOrders: number;
}

export interface TopProduct {
  productId: number;
  productName: string;
  totalSold: number;
  revenue: number;
}

export interface TopCustomer {
  userId: number;
  fullName: string;
  email: string;
  totalOrders: number;
  totalSpent: number;
}

export interface OverviewStats {
  totalRevenue: number;
  totalOrders: number;
  totalProducts: number;
  totalUsers: number;
  pendingOrders: number;
}

```

5. SERVICE LAYER

5.1 Axios Client với Interceptors

```

// services/api/axiosClient.ts
import axios, { AxiosInstance, InternalAxiosRequestConfig, AxiosError } from
'axios';
import { store } from '@app/store';
import { logout, setTokens } from '@features/auth/store/authSlice';
import { authService } from '@features/auth/services/authService';

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ||
'http://localhost:8080/api/v1';

const axiosClient: AxiosInstance = axios.create({
  baseURL: API_BASE_URL,
  headers: { 'Content-Type': 'application/json' },
  timeout: 15000,
});

// — Request Interceptor: đính kèm Access Token —
axiosClient.interceptors.request.use(
  (config: InternalAxiosRequestConfig) => {
    const token = store.getState().auth.accessToken;
    if (token && config.headers) {
      config.headers.Authorization = `Bearer ${token}`;
    }
  }
)

```

```

    return config;
  },
  (error) => Promise.reject(error)
);

// — Response Interceptor: xử lý 401 + Refresh Token —
let isRefreshing = false;
let failedQueue: Array<{ resolve: (value: string) => void; reject: (reason?: unknown) => void }> = [];

const processQueue = (error: Error | null, token: string | null = null) => {
  failedQueue.forEach((prom) => {
    if (error) prom.reject(error);
    else prom.resolve(token!);
  });
  failedQueue = [];
};

axiosClient.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as InternalAxiosRequestConfig & { _retry?: boolean };

    if (error.response?.status === 401 && !originalRequest._retry) {
      if (isRefreshing) {
        return new Promise((resolve, reject) => {
          failedQueue.push({ resolve, reject });
        }).then((token) => {
          originalRequest.headers.Authorization = `Bearer ${token}`;
          return axiosClient(originalRequest);
        });
      }

      originalRequest._retry = true;
      isRefreshing = true;

      const refreshToken = store.getState().auth.refreshToken;
      if (!refreshToken) {
        store.dispatch(logout());
        return Promise.reject(error);
      }

      try {
        const data = await authService.refreshToken({ refreshToken });
        store.dispatch(setTokens(data));
        processQueue(null, data.accessToken);
        originalRequest.headers.Authorization = `Bearer ${data.accessToken}`;
        return axiosClient(originalRequest);
      } catch (refreshError) {
        processQueue(refreshError as Error, null);
        store.dispatch(logout());
        return Promise.reject(refreshError);
      } finally {
        isRefreshing = false;
      }
    }

    return Promise.reject(error);
  }
);

export default axiosClient;

```

5.2 Auth Service

```
// features/auth/services/authService.ts
import axiosClient from '@services/api/axiosClient';
import type {
  LoginRequest, RegisterRequest, AuthResponse,
  RefreshTokenRequest, LogoutRequest,
  IntrospectRequest, IntrospectResponse, ChangePasswordRequest
} from '../types/auth.types';

export const authService = {
  login: (data: LoginRequest) =>
    axiosClient.post<{ result: AuthResponse }>('/auth/login', data).then(r =>
r.data.result),

  register: (data: RegisterRequest) =>
    axiosClient.post('/auth/register', data).then(r => r.data),

  logout: (data: LogoutRequest) =>
    axiosClient.post('/auth/logout', data).then(r => r.data),

  refreshToken: (data: RefreshTokenRequest) =>
    axiosClient.post<{ result: AuthResponse }>('/auth/refresh-token',
data).then(r => r.data.result),

  introspect: (data: IntrospectRequest) =>
    axiosClient.post<{ result: IntrospectResponse }>('/auth/introspect',
data).then(r => r.data.result),

  changePassword: (data: ChangePasswordRequest) =>
    axiosClient.put('/auth/change-password', data).then(r => r.data),
};
```

5.3 Product Service

```
// features/products/services/productService.ts
import axiosClient from '@services/api/axiosClient';
import type {
  Product, CreateProductRequest,
  UpdateProductRequest, ProductQueryParams
} from '../types/product.types';
import type { PaginationResponse } from '@types/common.types';

export const productService = {
  getAll: (params: ProductQueryParams) =>
    axiosClient.get<{ result: PaginationResponse<Product> }>('/products',
{ params })
    .then(r => r.data.result),

  getById: (id: number) =>
    axiosClient.get<{ result: Product }>(`/products/${id}`).then(r =>
r.data.result),

  search: (params: ProductQueryParams) =>
    axiosClient.get<{ result: PaginationResponse<Product> }>('/products/search',
{ params })
    .then(r => r.data.result),

  create: (data: CreateProductRequest) =>
    axiosClient.post<{ result: Product }>('/products', data).then(r =>
r.data.result),

  update: (id: number, data: UpdateProductRequest) =>
```

```

    axiosClient.put<{ result: Product }>(`/products/${id}`, data).then(r =>
r.data.result),

    delete: (id: number) =>
      axiosClient.delete(`/products/${id}`).then(r => r.data),

    uploadImages: (id: number, files: File[]) => {
      const formData = new FormData();
      files.forEach(file => formData.append('images', file));
      return axiosClient.post(`/products/${id}/images`, formData, {
        headers: { 'Content-Type': 'multipart/form-data' },
      }).then(r => r.data);
    },

    deleteImage: (productId: number, imageId: number) =>
      axiosClient.delete(`/products/${productId}/images/${imageId}`).then(r =>
r.data),
  };

```

5.4 Cart Service

```

// features/cart/services/cartService.ts
import axiosClient from '@services/api/axiosClient';
import type { Cart, AddToCartRequest, UpdateCartItemRequest } from
'../types/cart.types';

export const cartService = {
  getCart: () =>
    axiosClient.get<{ result: Cart }>('/cart').then(r => r.data.result),

  addItem: (data: AddToCartRequest) =>
    axiosClient.post<{ result: Cart }>('/cart/items', data).then(r =>
r.data.result),

  updateItem: (itemId: number, data: UpdateCartItemRequest) =>
    axiosClient.put<{ result: Cart }>(`/cart/items/${itemId}`, data).then(r =>
r.data.result),

  deleteItem: (itemId: number) =>
    axiosClient.delete<{ result: Cart }>(`/cart/items/${itemId}`).then(r =>
r.data.result),

  clearCart: () =>
    axiosClient.delete('/cart/clear').then(r => r.data),
};

```

5.5 Order Service

```

// features/orders/services/orderService.ts
import axiosClient from '@services/api/axiosClient';
import type {
  Order, CreateOrderRequest, CancelOrderRequest,
  UpdateOrderStatusRequest, OrderQueryParams
} from '../types/order.types';
import type { PaginationResponse } from '@types/common.types';

export const orderService = {
  createOrder: (data: CreateOrderRequest) =>
    axiosClient.post<{ result: Order }>('/orders', data).then(r =>
r.data.result),

  getMyOrders: (params: OrderQueryParams) =>

```

```

    axiosClient.get<{ result: PaginationResponse<Order> }>('/orders',
{ params })
    .then(r => r.data.result),

    getById: (id: number) =>
    axiosClient.get<{ result: Order }>(`/orders/${id}`).then(r =>
r.data.result),

    cancelOrder: (id: number, data: CancelOrderRequest) =>
    axiosClient.put(`/orders/${id}/cancel`, data).then(r => r.data),

    // Admin
    getAllOrders: (params: OrderQueryParams) =>
    axiosClient.get<{ result: PaginationResponse<Order> }>('/orders/admin/all',
{ params })
    .then(r => r.data.result),

    updateStatus: (id: number, data: UpdateOrderStatusRequest) =>
    axiosClient.put(`/orders/admin/${id}/status`, data).then(r => r.data),
};

```

6. STATE MANAGEMENT (Redux Toolkit)

```

// features/auth/store/authSlice.ts
import { createSlice, PayloadAction } from '@reduxjs/toolkit';
import type { UserRole } from '@features/users/types/user.types';

interface AuthState {
  accessToken: string | null;
  refreshToken: string | null;
  role: UserRole | null;
  email: string | null;
  isAuthenticated: boolean;
}

const loadFromStorage = (): Partial<AuthState> => {
  try {
    return {
      accessToken: localStorage.getItem('accessToken'),
      refreshToken: localStorage.getItem('refreshToken'),
      role: localStorage.getItem('role') as UserRole | null,
      email: localStorage.getItem('email'),
    };
  } catch { return {}; }
};

const initialState: AuthState = {
  accessToken: null,
  refreshToken: null,
  role: null,
  email: null,
  isAuthenticated: false,
  ...loadFromStorage(),
  // re-check isAuthenticated after loading
  get isAuthenticated() { return !!this.accessToken; },
};

export const authSlice = createSlice({
  name: 'auth',
  initialState: {
    ...initialState,

```

```

    isAuthenticated: !!initialState.accessToken,
  },
  reducers: {
    setTokens: (state, action: PayloadAction<{ accessToken: string;
refreshToken: string }>) => {
      state.accessToken = action.payload.accessToken;
      state.refreshToken = action.payload.refreshToken;
      state.isAuthenticated = true;
      localStorage.setItem('accessToken', action.payload.accessToken);
      localStorage.setItem('refreshToken', action.payload.refreshToken);
    },
    setUser: (state, action: PayloadAction<{ email: string; role: UserRole }>)
=> {
      state.email = action.payload.email;
      state.role = action.payload.role;
      localStorage.setItem('email', action.payload.email);
      localStorage.setItem('role', action.payload.role);
    },
    logout: (state) => {
      state.accessToken = null;
      state.refreshToken = null;
      state.email = null;
      state.role = null;
      state.isAuthenticated = false;
      localStorage.clear();
    },
  },
});

```

```

export const { setTokens, setUser, logout } = authSlice.actions;
export default authSlice.reducer;

```

```

// features/cart/store/cartSlice.ts
import { createSlice, PayloadAction } from '@reduxjs/toolkit';
import type { Cart } from '../types/cart.types';

```

```

interface CartState {
  cart: Cart | null;
  itemCount: number;
  isOpen: boolean;
}

```

```

const initialState: CartState = { cart: null, itemCount: 0, isOpen: false };

```

```

export const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: {
    setCart: (state, action: PayloadAction<Cart>) => {
      state.cart = action.payload;
      state.itemCount = action.payload.totalItems;
    },
    clearCart: (state) => {
      state.cart = null;
      state.itemCount = 0;
    },
    toggleCartDrawer: (state) => {
      state.isOpen = !state.isOpen;
    },
  },
});

```

```

export const { setCart, clearCart, toggleCartDrawer } = cartSlice.actions;

```

```
export default cartSlice.reducer;

// app/store.ts
import { configureStore } from '@reduxjs/toolkit';
import authReducer from '@features/auth/store/authSlice';
import cartReducer from '@features/cart/store/cartSlice';

export const store = configureStore({
  reducer: {
    auth: authReducer,
    cart: cartReducer,
  },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

State cần quản lý:

Redux (global, persist)	React Query (server state)	Local useState
auth.accessToken	Products list/detail	Form state
auth.refreshToken	Cart data	Modal open/close
auth.role	Orders	Filter/sort UI
cart.itemCount	Reviews	
cart.isOpen	User profile	

7. DANH SÁCH MÀN HÌNH & COMPONENTS

Customer App

Màn hình	Route	API gọi	Components
Trang chủ	/	GET /products (featured), GET /categories	CategoryTree, ProductGrid, ProductCard
Danh sách sản phẩm	/products	GET /products	ProductGrid, ProductCard, Pagination, FilterSidebar
Tìm kiếm	/products/search?keyword=	GET /products/search	ProductGrid, SearchBar, Pagination
Chi tiết sản phẩm	/products/:id	GET /products/:id, GET /reviews/products/:id	ProductImageGallery, ReviewList, ReviewForm, AddToCartButton
Giỏ hàng	/cart	GET /cart, PUT /cart/items/:id,	CartItem,

Màn hình	Route	API gọi	Components
		DELETE /cart/items/:id	CartSummary, QuantityControl
Checkout	/checkout	POST /orders	CheckoutForm, OrderSummary, AddressInput, PaymentMethodSelect
Đơn hàng của tôi	/orders	GET /orders	OrderCard, OrderStatusBadge, DateRangePicker, Pagination
Chi tiết đơn hàng	/orders/:id	GET /orders/:id	OrderDetail, OrderItems, OrderTimeline, CancelOrderButton
Hồ sơ cá nhân	/profile	GET /users/me, PUT /users/me, PUT /auth/change-password	ProfileForm, ChangePasswordForm
Đăng nhập	/login	POST /auth/login	LoginForm
Đăng ký	/register	POST /auth/register	RegisterForm

Admin Dashboard

Màn hình	Route	API gọi	Components
Dashboard	/admin	GET /admin/statistics/overview	OverviewCards, RevenueChart, TopProductsTable, TopCustomersTable
Quản lý sản phẩm	/admin/products	GET /products	Table, Pagination, SearchBar, ConfirmModal
Thêm/Sửa sản phẩm	/admin/products/new, /admin/products/:id/edit	POST/PUT /products, POST /products/:id/images	ProductForm, ImageUpload
Quản lý danh mục	/admin/categories	GET /categories, POST/PUT/DELETE /categories	CategoryTree, CategoryForm, Modal

Màn hình	Route	API gọi	Components
Quản lý đơn hàng	/admin/orders	GET /orders/admin/all	Table, OrderStatusBadge, StatusSelect, DateRangePicker
Quản lý người dùng	/admin/users	GET /users/admin/all, PUT lock/unlock	Table, UserStatusBadge, ConfirmModal

8. ROUTING

```
// routes/index.tsx
import { createBrowserRouter, RouterProvider } from 'react-router-dom';
import MainLayout from '@layouts/MainLayout';
import AdminLayout from '@layouts/AdminLayout';
import AuthLayout from '@layouts/AuthLayout';
import ProtectedRoute from '@components/common/ProtectedRoute';
import AdminRoute from '@components/common/AdminRoute';

// Public pages
import HomePage from '@pages/public/HomePage';
import ProductListPage from '@pages/public/ProductListPage';
import ProductDetailPage from '@pages/public/ProductDetailPage';
import SearchResultPage from '@pages/public/SearchResultPage';

// Auth pages
import LoginPage from '@pages/auth/LoginPage';
import RegisterPage from '@pages/auth/RegisterPage';

// User pages
import CartPage from '@pages/user/CartPage';
import CheckoutPage from '@pages/user/CheckoutPage';
import OrderListPage from '@pages/user/OrderListPage';
import OrderDetailPage from '@pages/user/OrderDetailPage';
import ProfilePage from '@pages/user/ProfilePage';

// Admin pages
import DashboardPage from '@pages/admin/DashboardPage';
import ProductManagePage from '@pages/admin/ProductManagePage';
import ProductFormPage from '@pages/admin/ProductFormPage';
import CategoryManagePage from '@pages/admin/CategoryManagePage';
import OrderManagePage from '@pages/admin/OrderManagePage';
import UserManagePage from '@pages/admin/UserManagePage';

const router = createBrowserRouter([
  // — Public Routes —————
  {
    element: <MainLayout />,
    children: [
      { path: '/', element: <HomePage /> },
      { path: '/products', element: <ProductListPage /> },
      { path: '/products/:id', element: <ProductDetailPage /> },
      { path: '/products/search', element: <SearchResultPage /> },
    ],
  },
],
```

```

// — Auth Routes —————
{
  element: <AuthLayout />,
  children: [
    { path: '/login', element: <LoginPage /> },
    { path: '/register', element: <RegisterPage /> },
  ],
},

// — Protected User Routes —————
{
  element: <ProtectedRoute><MainLayout /></ProtectedRoute>,
  children: [
    { path: '/cart', element: <CartPage /> },
    { path: '/checkout', element: <CheckoutPage /> },
    { path: '/orders', element: <OrderListPage /> },
    { path: '/orders/:id', element: <OrderDetailPage /> },
    { path: '/profile', element: <ProfilePage /> },
  ],
},

// — Admin Routes —————
{
  element: <AdminRoute><AdminLayout /></AdminRoute>,
  path: '/admin',
  children: [
    { index: true, element: <DashboardPage /> },
    { path: 'products', element: <ProductManagePage /> },
    { path: 'products/new', element: <ProductFormPage /> },
    { path: 'products/:id/edit', element: <ProductFormPage /> },
    { path: 'categories', element: <CategoryManagePage /> },
    { path: 'orders', element: <OrderManagePage /> },
    { path: 'users', element: <UserManagePage /> },
  ],
},
]);

export default function AppRouter() {
  return <RouterProvider router={router} />;
}

// components/common/ProtectedRoute.tsx
import { Navigate, useLocation } from 'react-router-dom';
import { useAppSelector } from '@app/hooks';

export default function ProtectedRoute({ children }: { children: React.ReactNode }) {
  const { isAuthenticated } = useAppSelector((s) => s.auth);
  const location = useLocation();
  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }
  return <>{children}</>;
}

// components/common/AdminRoute.tsx
export default function AdminRoute({ children }: { children: React.ReactNode }) {
  const { isAuthenticated, role } = useAppSelector((s) => s.auth);
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  if (role !== 'ROLE_ADMIN') return <Navigate to="/" replace />;
  return <>{children}</>;
}

```

9. FORM VALIDATION (Zod Schemas)

```
// features/auth/schemas.ts
import { z } from 'zod';

export const loginSchema = z.object({
  email: z.string().email('Email không hợp lệ'),
  password: z.string().min(6, 'Mật khẩu ít nhất 6 ký tự'),
});

export const registerSchema = z.object({
  fullName: z.string().min(2, 'Họ tên ít nhất 2 ký tự').max(100),
  email: z.string().email('Email không hợp lệ'),
  password: z
    .string()
    .min(8, 'Mật khẩu ít nhất 8 ký tự')
    .regex(/[A-Z]/, 'Cần ít nhất 1 chữ hoa')
    .regex(/[0-9]/, 'Cần ít nhất 1 chữ số'),
  phone: z.string().regex(/^\d{10,11}$/, 'Số điện thoại không hợp lệ'),
});

export const changePasswordSchema = z.object({
  email: z.string().email(),
  oldPassword: z.string().min(1, 'Vui lòng nhập mật khẩu cũ'),
  newPassword: z
    .string()
    .min(8, 'Mật khẩu mới ít nhất 8 ký tự')
    .regex(/[A-Z]/, 'Cần ít nhất 1 chữ hoa')
    .regex(/[0-9]/, 'Cần ít nhất 1 chữ số'),
}).refine(d => d.oldPassword !== d.newPassword, {
  message: 'Mật khẩu mới phải khác mật khẩu cũ',
  path: ['newPassword'],
});

// features/products/schemas.ts
export const productSchema = z.object({
  name: z.string().min(3, 'Tên sản phẩm ít nhất 3 ký tự').max(255),
  description: z.string().min(10, 'Mô tả ít nhất 10 ký tự'),
  price: z.number().positive('Giá phải lớn hơn 0'),
  stockQuantity: z.number().int().min(0, 'Số lượng không âm'),
  categoryId: z.number({ required_error: 'Vui lòng chọn danh mục' }),
});

// features/orders/schemas.ts
export const checkoutSchema = z.object({
  shippingAddress: z.string().min(10, 'Địa chỉ quá ngắn').max(500),
  note: z.string().max(500).optional(),
  paymentMethod: z.enum(['COD', 'BANK_TRANSFER', 'MOMO']),
});

// features/orders/schemas.ts (cancel)
export const cancelOrderSchema = z.object({
  reason: z.string().min(5, 'Vui lòng mô tả lý do hủy').max(500),
});

// features/reviews/schemas.ts
export const reviewSchema = z.object({
  rating: z.number().int().min(1, 'Tối thiểu 1 sao').max(5, 'Tối đa 5 sao'),
  comment: z.string().min(5, 'Nhận xét quá ngắn').max(1000),
});
```

```
// features/users/schemas.ts
export const updateProfileSchema = z.object({
  fullName: z.string().min(2).max(100),
  phone: z.string().regex(/^d{10,11}$/, 'Số điện thoại không hợp lệ'),
  address: z.string().max(500).optional(),
});
```

10. REACT QUERY STRATEGY

```
// constants/queryKeys.ts
export const QUERY_KEYS = {
  // Products
  products: (params?: object) => ['products', params] as const,
  product: (id: number) => ['products', id] as const,
  productSearch: (params?: object) => ['products', 'search', params] as const,

  // Categories
  categories: () => ['categories'] as const,
  category: (id: number) => ['categories', id] as const,

  // Cart
  cart: () => ['cart'] as const,

  // Orders
  myOrders: (params?: object) => ['orders', 'mine', params] as const,
  order: (id: number) => ['orders', id] as const,
  adminOrders: (params?: object) => ['orders', 'admin', params] as const,

  // Reviews
  reviews: (productId: number) => ['reviews', productId] as const,

  // Users
  me: () => ['users', 'me'] as const,
  adminUsers: () => ['users', 'admin', 'all'] as const,

  // Statistics
  statsOverview: () => ['statistics', 'overview'] as const,
  statsRevenue: (params?: object) => ['statistics', 'revenue', params] as const,
  statsTopProducts: (limit: number) => ['statistics', 'top-products', limit] as
const,
  statsTopCustomers: (limit: number) => ['statistics', 'top-customers', limit]
as const,
} as const;

// features/products/hooks/useProducts.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { productService } from '../../services/productService';
import { QUERY_KEYS } from '@constants/queryKeys';
import type { ProductQueryParams } from '../../types/product.types';

export const useProducts = (params: ProductQueryParams) =>
  useQuery({
    queryKey: QUERY_KEYS.products(params),
    queryFn: () => productService.getAll(params),
    staleTime: 5 * 60 * 1000, // 5 phút
    gcTime: 10 * 60 * 1000, // 10 phút
    placeholderData: (prev) => prev, // giữ data cũ khi đổi page
  });

export const useProduct = (id: number) =>
```

```

    useQuery({
      queryKey: QUERY_KEYS.product(id),
      queryFn: () => productService.getById(id),
      staleTime: 5 * 60 * 1000,
      enabled: !!id,
    });

export const useCreateProduct = () => {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: productService.create,
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ['products'] });
    },
  });
};

export const useUpdateProduct = (id: number) => {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (data: Parameters<typeof productService.update>[1]) =>
      productService.update(id, data),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ['products'] });
      qc.invalidateQueries({ queryKey: QUERY_KEYS.product(id) });
    },
  });
};

export const useDeleteProduct = () => {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: productService.delete,
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ['products'] });
    },
  });
};

// features/cart/hooks/useCart.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { cartService } from '../services/cartService';
import { QUERY_KEYS } from '@/constants/queryKeys';
import { useAppDispatch } from '@/app/hooks';
import { setCart } from '../store/cartSlice';

export const useCart = () => {
  const dispatch = useAppDispatch();
  return useQuery({
    queryKey: QUERY_KEYS.cart(),
    queryFn: async () => {
      const cart = await cartService.getCart();
      dispatch(setCart(cart));
      return cart;
    },
    staleTime: 30 * 1000, // 30 giây
  });
};

export const useAddToCart = () => {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: cartService.addItem,
  });
};

```

```

    onSuccess: (cart) => {
      qc.setQueryData(QUERY_KEYS.cart(), cart);
    },
  });
};

export const useUpdateCartItem = () => {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ itemId, data }: { itemId: number; data: { quantity:
number } }) =>
      cartService.updateItem(itemId, data),
    onSuccess: (cart) => qc.setQueryData(QUERY_KEYS.cart(), cart),
  });
};

```

Cache Strategy tóm tắt:

Data	staleTime	gcTime	Invalidate khi
Products list	5 phút	10 phút	Create/Update/Delete product
Product detail	5 phút	10 phút	Update/Delete product
Cart	30 giây	5 phút	Add/Update/Delete/Checkout
Orders	1 phút	5 phút	Create/Cancel order
Reviews	3 phút	10 phút	Create/Update/Delete review
Statistics	10 phút	30 phút	Manual refetch
Categories	10 phút	30 phút	CRUD category

11. AUTHENTICATION FLOW

11.1 Login Flow

User nhập email/password

- Validate bằng Zod
- POST /auth/login
- Nhận { accessToken, refreshToken }
- Decode JWT: lấy email, scope (role)
- dispatch setTokens() + setUser()
- Lưu vào localStorage
- Navigate to "/" hoặc "/admin" tùy role

11.2 Logout Flow

User click Logout

- Lấy accessToken + refreshToken từ store
- POST /auth/logout { accessToken, refreshToken }
- dispatch logout() → xóa store + localStorage
- Navigate to "/login"

11.3 Refresh Token Flow (tự động)

Request bị 401

- Axios interceptor bắt lỗi
- isRefreshing = true, queue các request đang chờ
- POST /auth/refresh-token { refreshToken }
- Nhận accessToken mới
- dispatch setTokens()
- Retry tất cả request trong queue
- Nếu refresh cũng thất bại → dispatch logout() → redirect /login

11.4 Persist Login

```
// Khi app khởi động (main.tsx hoặc App.tsx):
const storedToken = localStorage.getItem('accessToken');
if (storedToken) {
  // Introspect để verify token còn valid
  authService.introspect({ accessToken: storedToken })
    .then(({ valid }) => {
      if (!valid) store.dispatch(logout());
    })
    .catch(() => store.dispatch(logout()));
}
```

11.5 Decode JWT để lấy Role

```
// utils/tokenUtils.ts
export function decodeJwt(token: string): { sub: string; scope: string; exp:
number } | null {
  try {
    const payload = token.split('.')[1];
    const decoded = JSON.parse(atob(payload));
    return decoded;
  } catch {
    return null;
  }
}
```

12. CODE MẪU QUAN TRỌNG

12.1 Login Page (React Hook Form + Zod)

```
// pages/auth/LoginPage.tsx
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { useMutation } from '@tanstack/react-query';
import { useNavigate, useLocation } from 'react-router-dom';
import { z } from 'zod';
import { authService } from '@features/auth/services/authService';
import { useAppDispatch } from '@app/hooks';
import { setTokens, setUser } from '@features/auth/store/authSlice';
import { decodeJwt } from '@utils/tokenUtils';
import { loginSchema } from '@features/auth/schemas';

type LoginForm = z.infer<typeof loginSchema>;

export default function LoginPage() {
  const dispatch = useAppDispatch();
```

```

const navigate = useNavigate();
const location = useLocation();
const from = location.state?.from?.pathname || '/';

const { register, handleSubmit, formState: { errors } } = useForm<LoginForm>({
  resolver: zodResolver(loginSchema),
});

const { mutate, isPending, error } = useMutation({
  mutationFn: authService.login,
  onSuccess: (data) => {
    dispatch(setTokens(data));
    const decoded = decodeJwt(data.accessToken);
    if (decoded) {
      dispatch(setUser({ email: decoded.sub, role: decoded.scope as any }));
    }
    const isAdmin = decoded?.scope === 'ROLE_ADMIN';
    navigate(isAdmin ? '/admin' : from, { replace: true });
  },
});

return (
  <form onSubmit={handleSubmit((d) => mutate(d))}>
    <input {...register('email')} placeholder="Email" />
    {errors.email && <span>{errors.email.message}</span>}

    <input {...register('password')} type="password" placeholder="Mật khẩu" />
    {errors.password && <span>{errors.password.message}</span>}

    {error && <div>Email hoặc mật khẩu không đúng</div>}

    <button type="submit" disabled={isPending}>
      {isPending ? 'Đang đăng nhập...' : 'Đăng nhập'}
    </button>
  </form>
);
}

```

12.2 Add To Cart với Optimistic Update

```

// features/products/components/ProductDetail.tsx (snippet)
import { useAddToCart } from '@features/cart/hooks/useCart';
import { useAppDispatch } from '@app/hooks';
import { toggleCartDrawer } from '@features/cart/store/cartSlice';
import { useState } from 'react';

function AddToCartButton({ productId }: { productId: number }) {
  const [quantity, setQuantity] = useState(1);
  const dispatch = useAppDispatch();
  const { mutate, isPending } = useAddToCart();

  const handleAdd = () => {
    mutate({ productId, quantity }, {
      onSuccess: () => dispatch(toggleCartDrawer()),
    });
  };

  return (
    <div>
      <input type="number" min={1} value={quantity}
        onChange={(e) => setQuantity(Number(e.target.value))} />
      <button onClick={handleAdd} disabled={isPending}>
        {isPending ? 'Đang thêm...' : 'Thêm vào giỏ'}
      </button>
    </div>
  );
}

```

```

    </button>
  </div>
);
}

```

12.3 Admin Order Status Update

```

// features/orders/components/OrderStatusSelect.tsx
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { orderService } from '../../services/orderService';
import { QUERY_KEYS } from '@constants/queryKeys';
import type { OrderStatus } from '../../types/order.types';

const STATUS_OPTIONS: OrderStatus[] = ['PENDING', 'CONFIRMED', 'SHIPPING',
'DELIVERED', 'CANCELLED'];

export default function OrderStatusSelect({ orderId, currentStatus }: {
  orderId: number; currentStatus: OrderStatus;
}) {
  const qc = useQueryClient();
  const { mutate, isPending } = useMutation({
    mutationFn: (newStatus: OrderStatus) =>
      orderService.updateStatus(orderId, { newStatus }),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: QUERY_KEYS.adminOrders() });
      qc.invalidateQueries({ queryKey: QUERY_KEYS.order(orderId) });
    },
  });

  return (
    <select
      value={currentStatus}
      onChange={(e) => mutate(e.target.value as OrderStatus)}
      disabled={isPending}
    >
      {STATUS_OPTIONS.map(s => <option key={s} value={s}>{s}</option>)}
    </select>
  );
}

```

12.4 Product Search với Debounce

```

// features/products/hooks/useProductSearch.ts
import { useState } from 'react';
import { useQuery } from '@tanstack/react-query';
import { useDebounce } from '@hooks/useDebounce';
import { productService } from '../../services/productService';
import { QUERY_KEYS } from '@constants/queryKeys';

export function useProductSearch() {
  const [keyword, setKeyword] = useState('');
  const [page, setPage] = useState(0);
  const debouncedKeyword = useDebounce(keyword, 400);

  const params = { keyword: debouncedKeyword, page, size: 12, sort: 'newest' as
const };

  const query = useQuery({
    queryKey: QUERY_KEYS.productSearch(params),
    queryFn: () => productService.search(params),
    enabled: debouncedKeyword.length >= 2,
    staleTime: 2 * 60 * 1000,
  });
}

```

```
});  
  
return { keyword, setKeyword, page, setPage, ...query };  
}
```

13. CHECKLIST TRIỂN KHAI FRONTEND

Phase 1 – Cài đặt & Scaffold

- ☐ Khởi tạo Vite + React + TypeScript
- ☐ Cài dependencies: axios react-router-dom @tanstack/react-query @reduxjs/toolkit react-redux react-hook-form zod @hookform/resolvers tailwindcss
- ☐ Cấu hình tsconfig.json với path aliases (@/)
- ☐ Setup TailwindCSS
- ☐ Tạo cấu trúc thư mục như trên
- ☐ Cấu hình VITE_API_BASE_URL trong .env

Phase 2 – Core Infrastructure

- ☐ Tạo axiosClient.ts với interceptors
- ☐ Tạo Redux store (authSlice, cartSlice)
- ☐ Wrap app với QueryClientProvider + Provider (Redux) + RouterProvider
- ☐ Implement ProtectedRoute + AdminRoute
- ☐ Implement tokenUtils.ts (decode JWT)

Phase 3 – Authentication Module

- ☐ LoginPage với form validation
- ☐ RegisterPage với form validation
- ☐ logout flow
- ☐ Persist login (localStorage + introspect on startup)
- ☐ Refresh token interceptor

Phase 4 – Public Pages (Customer)

- ☐ HomePage – featured products + categories
- ☐ ProductListPage – danh sách + phân trang + filter sort
- ☐ ProductDetailPage – chi tiết + ảnh gallery + reviews

- ☐ SearchResultPage – tìm kiếm debounce

Phase 5 – Cart & Checkout

- ☐ CartPage – xem, cập nhật, xóa
- ☐ CartDrawer – slide-out cart
- ☐ CheckoutPage – form + tạo đơn

Phase 6 – Orders & Profile

- ☐ OrderListPage – filter by status + date range
- ☐ OrderDetailPage – timeline + cancel
- ☐ ProfilePage – update info + đổi mật khẩu
- ☐ ReviewForm – sau khi nhận hàng (DELIVERED)

Phase 7 – Admin Dashboard

- ☐ AdminLayout với sidebar navigation
- ☐ DashboardPage – overview cards + revenue chart
- ☐ ProductManagePage – CRUD table
- ☐ ProductFormPage – form + image upload (multipart)
- ☐ CategoryManagePage – tree view CRUD
- ☐ OrderManagePage – filter + update status
- ☐ UserManagePage – list + lock/unlock

Phase 8 – Polish & QA

- ☐ Error boundaries
- ☐ Loading states (skeletons)
- ☐ Toast notifications (success/error)
- ☐ Responsive mobile
- ☐ 404 page
- ☐ Empty states
- ☐ Confirm modals cho destructive actions

14. CONSTANTS

```
// constants/app.ts
export const ORDER_STATUS = {
  PENDING: { label: 'Chờ xác nhận', color: 'yellow' },
```

```
    CONFIRMED: { label: 'Đã xác nhận', color: 'blue' },
    SHIPPING: { label: 'Đang giao', color: 'purple' },
    DELIVERED: { label: 'Đã giao', color: 'green' },
    CANCELLED: { label: 'Đã hủy', color: 'red' },
  } as const;

export const SORT_OPTIONS = [
  { value: 'newest', label: 'Mới nhất' },
  { value: 'oldest', label: 'Cũ nhất' },
  { value: 'price_asc', label: 'Giá tăng dần' },
  { value: 'price_desc', label: 'Giá giảm dần' },
] as const;

export const PAYMENT_METHODS = [
  { value: 'COD', label: 'Thanh toán khi nhận hàng' },
  { value: 'BANK_TRANSFER', label: 'Chuyển khoản ngân hàng' },
  { value: 'MOMO', label: 'Ví MoMo' },
] as const;

export const PAGE_SIZE_OPTIONS = [10, 20, 50] as const;
export const DEFAULT_PAGE_SIZE = 10;
```

Tài liệu được sinh tự động từ Bruno Collection shop-vibe (exported 2026-06-09). Cập nhật khi có thay đổi API.